BSc (Hons) in **Information Technology**

Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.* Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A mathematically perfect Table-Driven Agent is impossible to build for complex environments such as Chess because it would need to store an action for every possible percept sequence.
As the agent's lifetime increases, the number of possible percept sequences grows extremely quickly. This is known as combinatorial explosion.
Because the number of possible histories becomes enormous, the table would require an unrealistic amount of memory and computation. Therefore, in complex environments, we use agent programs instead of storing every possible percept sequence in a table.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
if percept['food_here']:
    return 'Suck'
elif percept['wall_ahead']:
    return 'TurnLeft'
else:
    return 'Forward'
```

If food_here is true, the agent performs Suck.
If wall_ahead is true, the agent performs TurnLeft.
Otherwise, the agent performs Forward.

The Simple Reflex Agent makes its decision only from the current percept and does not use any memory or percept history.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The Simple Reflex Agent gets stuck because the environment is partially observable and the agent has no percept history.
The agent only receives local information such as whether there is food at the current location or whether there is a wall ahead. It does not know its exact global position or the complete structure of the environment.
Because it has no memory, it cannot remember which cells it has already visited or whether it has already encountered the same situation before.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The Model-Based Agent maintains an internal state so that it can remember information from previous percepts and actions.
The Transition Model updates the internal state based on the previous action. For example, if the previous action was Forward, the agent updates its relative position. If the previous action was TurnLeft or TurnRight, it updates its internal facing direction.
The Sensor Model uses the current percept to update the agent’s knowledge of the environment. For example, if wall_ahead is true, the agent records the wall in its internal memory.
The agent also remembers visited cells and known walls. Before choosing a new action, it first updates its internal state and then checks this memory.
Because of this internal state, the Model-Based Agent can recognize that it has already visited certain locations and can select a different route instead of repeatedly following the same path.

Finalize and Lock Lab 02 Documentation